CNN-BASED PCB DEFECT DETECTION SYSTEM USING TRANSFER LEARNING

1. INTRODUCTION

Electronic circuit boards, commonly known as Printed Circuit Boards (PCBs), are important components of electronic devices. During manufacturing, PCBs may develop defects such as missing components, broken connections, short circuits, damaged tracks, and incorrect component placement.

Traditional PCB inspection is usually performed manually or using conventional inspection systems. Manual inspection can be time-consuming, repetitive, and prone to human errors. Computer vision and deep learning can be used to automate PCB inspection by identifying defective boards from captured images.

This project presents a **CNN-Based PCB Defect Detection System using Transfer Learning**. The system uses PCB images containing defective and non-defective samples for training and validation.

Instead of training a CNN completely from the beginning, the project uses **MobileNetV2**, a CNN architecture pretrained on ImageNet. Transfer learning allows the model to reuse previously learned visual features such as edges, shapes, textures, and patterns.

The trained model is then used to classify a PCB image as **Defective** or **Non-Defective**. The system displays the predicted class and confidence score.

Project Objectives

The main objectives are:

1. To develop an automated PCB defect detection system using deep learning.
2. To use a PCB image dataset for training and validation.
3. To implement CNN-based transfer learning using MobileNetV2.
4. To classify PCB images into defective and non-defective categories.
5. To provide image-based PCB inspection.
6. To display the predicted class and confidence score.
7. To explore the use of AI-based computer vision for automated manufacturing inspection.

2. TOOLS AND TECHNIQUES

2.1 Software Tools

Tool/Technology	Purpose
Python	Programming language
Google Colab	Development and training environment
TensorFlow	Deep learning framework
Keras	Neural network implementation
MobileNetV2	Transfer learning model
NumPy	Numerical computation
Matplotlib	Data visualization
Seaborn	Confusion matrix visualization
Scikit-learn	Model evaluation
PIL	Image processing

2.2 Hardware Requirements

The system requires:

- Computer/laptop
- Minimum 8 GB RAM recommended
- Internet connection
- PCB images captured using a camera or available dataset
- GPU for faster model training

Google Colab GPU can be used instead of a local GPU.

2.3 PCB Defect Dataset

The project uses a PCB image dataset containing images of circuit boards.

The dataset is divided into two categories:

Class	Examples
Defective	PCB containing visible manufacturing defects
Non-Defective	PCB without visible defects

The dataset is divided into **training and validation sets using an 80:20 split**.

2.4 Convolutional Neural Network

A Convolutional Neural Network (CNN) is a deep learning architecture commonly used for image-related tasks.

A CNN automatically learns visual features such as:

- Edges
- Shapes
- Textures
- Patterns
- Component structures

The general process is:

Input PCB Image
↓
Convolution
↓
Feature Extraction
↓
Pooling
↓
Deep Features
↓
Classification

CNNs are useful for PCB inspection because defective and non-defective circuit boards contain different visual characteristics.

2.5 Transfer Learning

Training a deep CNN from scratch requires a large amount of data and computational resources. Transfer learning provides an efficient alternative.

In transfer learning, a model that has already learned useful visual features from a large dataset is reused for a new task.

In this project, **MobileNetV2 pretrained on ImageNet** is used as the base model.

The pretrained model acts as a feature extractor, while a new classification layer is added for the two PCB categories.

2.6 MobileNetV2

MobileNetV2 is a lightweight CNN architecture designed to provide efficient image processing with relatively low computational requirements.

The advantages of MobileNetV2 for this project include:

- Lightweight architecture
- Faster training
- Reduced computational requirements
- Good image feature extraction capability
- Suitable for potential industrial edge-device deployment

3. METHODOLOGY AND IMPLEMENTATION

3.1 Data Collection

The PCB defect dataset is collected and organized into two class folders:

PCB Dataset
- Defective
- Non-Defective

3.2 Image Preprocessing

Before feeding the images to the model, preprocessing is performed.

The images are resized to:

224 × 224 pixels

The images are converted into RGB format and normalized according to the preprocessing requirements of MobileNetV2.

This ensures that all images have a consistent input format.

3.3 Data Augmentation

Data augmentation is applied to improve model generalization.

The following transformations are used:

- Random horizontal flipping
- Random rotation
- Random zoom
- Random contrast adjustment

3.4 Model Development

MobileNetV2 is loaded with pretrained ImageNet weights.

The original classification layer is removed, and a new classification head is added.

The architecture used is:

Input PCB Image
224 × 224 × 3
↓
Data Augmentation
↓
MobileNetV2
↓
Global Average Pooling
↓
Dropout
↓
Dense Layer
↓
Softmax
↓
2 PCB Classes

3.5 Classification Layer

The final Dense layer contains **two output neurons** because the dataset contains two classes.

The Softmax activation function converts the model's output into probabilities.

For example:

Defective : 0.91
Non-Defective : 0.09

The class with the highest probability is selected.

In this example:

Predicted Class = Defective
Confidence = 91%

3.6 Model Training

The model is trained using the following configuration:

Parameter	Value
Model	MobileNetV2
Input Size	224 × 224 × 3
Batch Size	32
Epochs	15
Optimizer	Adam
Learning Rate	0.0001
Loss Function	Binary/Categorical Crossentropy
Output Classes	2
Training/Validation Split	80:20

The pretrained MobileNetV2 layers are initially frozen, allowing the newly added classification layer to learn the characteristics of the PCB classes.

3.7 Model Evaluation

The model is evaluated using:

- **Accuracy** – Measures the proportion of correctly classified PCB images.
- **Precision** – Measures how many images predicted as defective actually belong to the defective class.
- **Recall** – Measures how many defective PCB images were correctly identified.
- **F1-Score** – Provides a combined measure of precision and recall.
- **Confusion Matrix** – Shows correct and incorrect predictions for each class.

4. CODING

4.1 Importing Libraries

```
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input

from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
```

4.2 Loading the Dataset

```
dataset_dir = "/content/PCB_Dataset"

train_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.20,
    subset="training",
    seed=42,
    image_size=(224, 224),
    batch_size=32,
    label_mode="categorical"
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    dataset_dir,
    validation_split=0.20,
    subset="validation",
    seed=42,
    image_size=(224, 224),
    batch_size=32,
    label_mode="categorical"
)

class_names = train_ds.class_names
print(class_names)
```

The expected class names are:

```
['Defective', 'Non-Defective']
```

4.3 Data Augmentation

```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.15),
    layers.RandomZoom(0.15),
    layers.RandomContrast(0.10)
])
```

4.4 MobileNetV2

```
base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights="imagenet"
)

base_model.trainable = False
```

4.5 Model Architecture

```
inputs = keras.Input(shape=(224, 224, 3))

x = data_augmentation(inputs)
x = preprocess_input(x)

x = base_model(
    x,
    training=False
)

x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)

outputs = layers.Dense(
    2,
    activation="softmax"
)(x)

model = keras.Model(
    inputs,
    outputs
)
```

4.6 Model Compilation

```
model.compile(
    optimizer=keras.optimizers.Adam(
        learning_rate=0.0001
    ),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

4.7 Model Training

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15
)
```

4.8 Accuracy Graph

```
plt.figure(figsize=(10,5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
plt.grid()
plt.show()
```

4.9 Confusion Matrix

```
y_true = []
y_pred = []

for images, labels in val_ds:
    predictions = model.predict(images, verbose=0)

    y_true.extend(np.argmax(labels.numpy(), axis=1))
    y_pred.extend(np.argmax(predictions, axis=1))

cm = confusion_matrix(
    y_true,
    y_pred
)

plt.figure(figsize=(7,6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=class_names,
```

```
    yticklabels=class_names
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

4.10 Prediction Function

```
from PIL import Image

def predict_pcb(image):

    image = image.convert("RGB")
    image = image.resize((224, 224))

    img_array = np.array(image)

    img_array = np.expand_dims(
        img_array,
        axis=0
    )

    prediction = model.predict(
        img_array,
        verbose=0
    )[0]

    predicted_index = np.argmax(prediction)

    predicted_class = class_names[
        predicted_index
    ]

    confidence = (
        prediction[predicted_index] * 100
    )

    return predicted_class, confidence
```

4.11 Model Saving

```
model.save(
    "/content/pcb_defect_detection_model.keras"
)

print("Model saved successfully!")
```

5. OUTPUT

The system produces several outputs during the implementation.

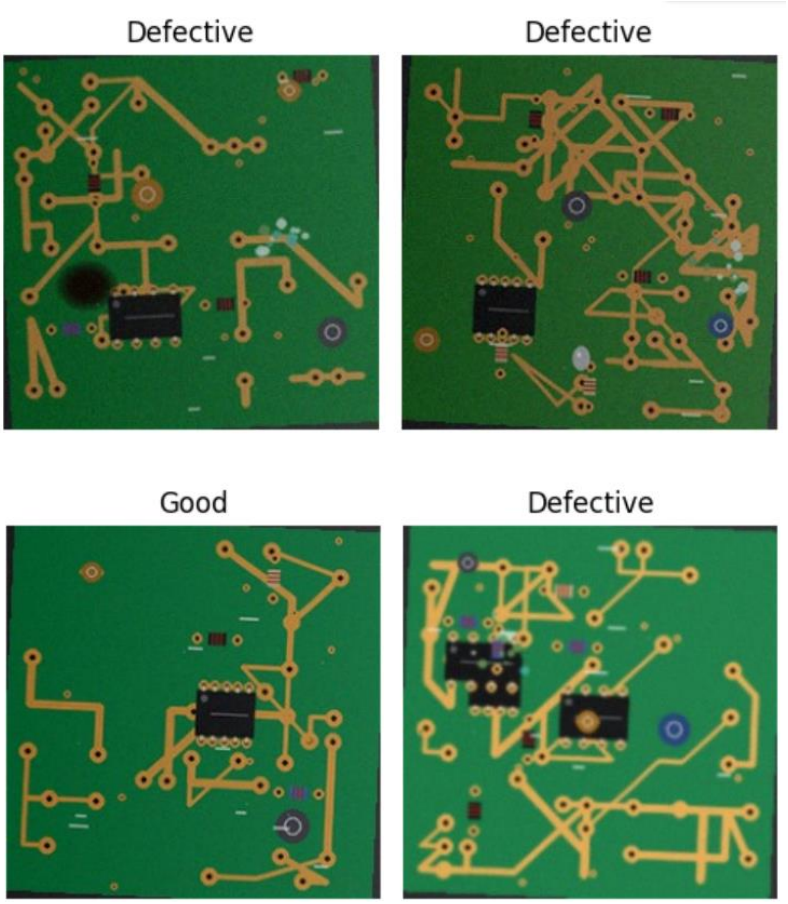
```
PCB DEFECT DETECTION SYSTEM

----------------------------
Image         : Defect_Detected_4-2.webp
Detected PCB  : Defective
Confidence    : 65.18 %


Model saved successfully!
```

5.1 Dataset Visualization

The PCB dataset images and their corresponding classes are displayed.



5.3 Model Architecture

The MobileNetV2-based PCB defect classification architecture is displayed.

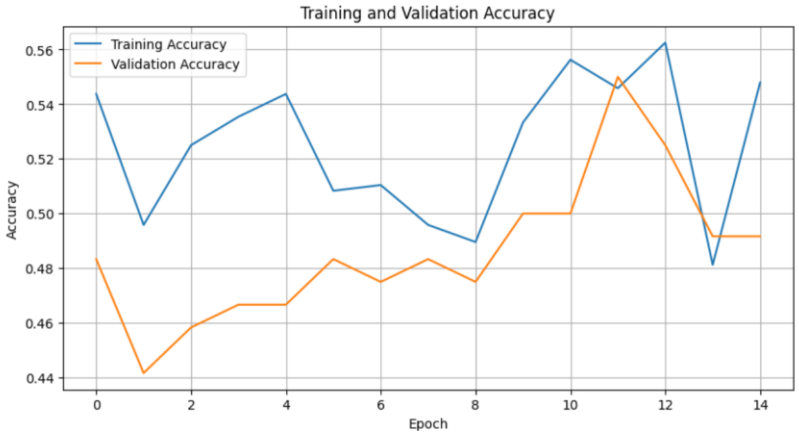
Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
sequential (Sequential)	(None, 224, 224, 3)	0
true_divide (TrueDivide)	(None, 224, 224, 3)	0
subtract (Subtract)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 2)	2,562

Total params: 2,260,546 (8.62 MB)
Trainable params: 2,562 (10.01 KB)
Non-trainable params: 2,257,984 (8.61 MB)

5.4 Training Accuracy

The training and validation accuracy graph is displayed to understand the learning performance of the model.



5.5 Training Loss

The training and validation loss graph is displayed to observe the loss reduction during training.



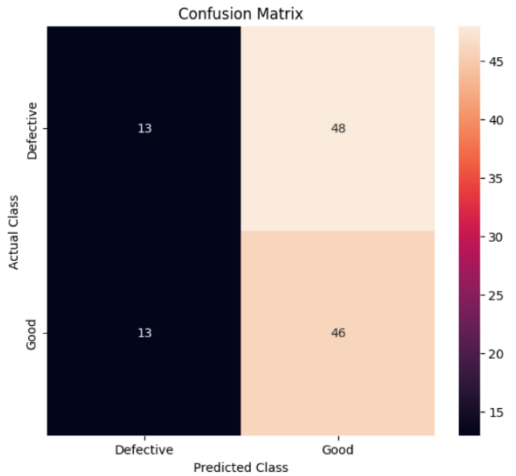
5.6 Classification Report

The classification report displays:

CLASSIFICATION REPORT	precision	recall	f1-score	support
Defective	0.50	0.21	0.30	61
Good	0.49	0.78	0.60	59
accuracy			0.49	120
macro avg	0.49	0.50	0.45	120
weighted avg	0.49	0.49	0.45	120

5.7 Confusion Matrix

The confusion matrix displays the number of correctly and incorrectly classified PCB images.



6. RESULT AND CONCLUSION

6.1 Result Analysis

The accuracy and loss graphs are used to understand the learning behavior of the model.

The confusion matrix provides a class-wise view of the model's predictions. It helps identify correctly classified PCB images and images that are incorrectly classified.

Some PCB defects may have similar visual characteristics to normal PCB patterns. Small defects, damaged tracks, missing components, and minor manufacturing variations may therefore be difficult for the model to recognize.

The performance can also be affected by image quality, lighting conditions, camera angle, PCB orientation, and the number of defect samples available in the dataset.

Therefore, the current model can be considered a **prototype PCB classification system** rather than a complete industrial automated optical inspection system.

6.2 Conclusion

This project successfully demonstrates the application of deep learning and transfer learning for automated PCB defect detection.

The combination of a PCB image dataset and MobileNetV2 provides a practical approach for classifying PCB images into defective and non-defective categories.

The CNN model automatically learns important visual features from PCB images and uses these features to perform classification.

The project demonstrates how AI-based computer vision can potentially assist manufacturing companies in reducing manual inspection effort and improving the consistency of PCB quality inspection.

7. FUTURE SCOPE

The project can be extended in several ways.

1. Multi-Defect Detection

The current system mainly classifies a PCB as defective or non-defective. A future system can identify the **exact type and location of the defect**.

2. Real-Time PCB Inspection

The system can be extended to continuously process images from a production-line camera.

3. Object Detection

Object detection models such as YOLO can be used to locate individual defects on the PCB.

For example:

PCB Image
↓
Defect Detection
↓
Defect Location
↓
Defect Type

4. Automated Production-Line Inspection

The AI model can be integrated with:

- Industrial cameras
- Conveyor belts
- Sensors
- Raspberry Pi
- NVIDIA Jetson
- PLC systems

This can allow defective PCBs to be automatically identified and separated.

5. Defect Classification

Future versions can identify different types of PCB defects such as:

- Missing component
- Broken track

- Short circuit
- Open circuit
- Incorrect component placement
- Damaged component

6. Web Application

A web application can allow users to upload PCB images and receive:

Defect Status
Confidence Score
Defect Type
Inspection Result

7. Edge AI Deployment

The model can potentially be optimized and deployed on edge devices such as **Raspberry Pi or NVIDIA Jetson** for real-time PCB inspection with reduced dependence on cloud computing.